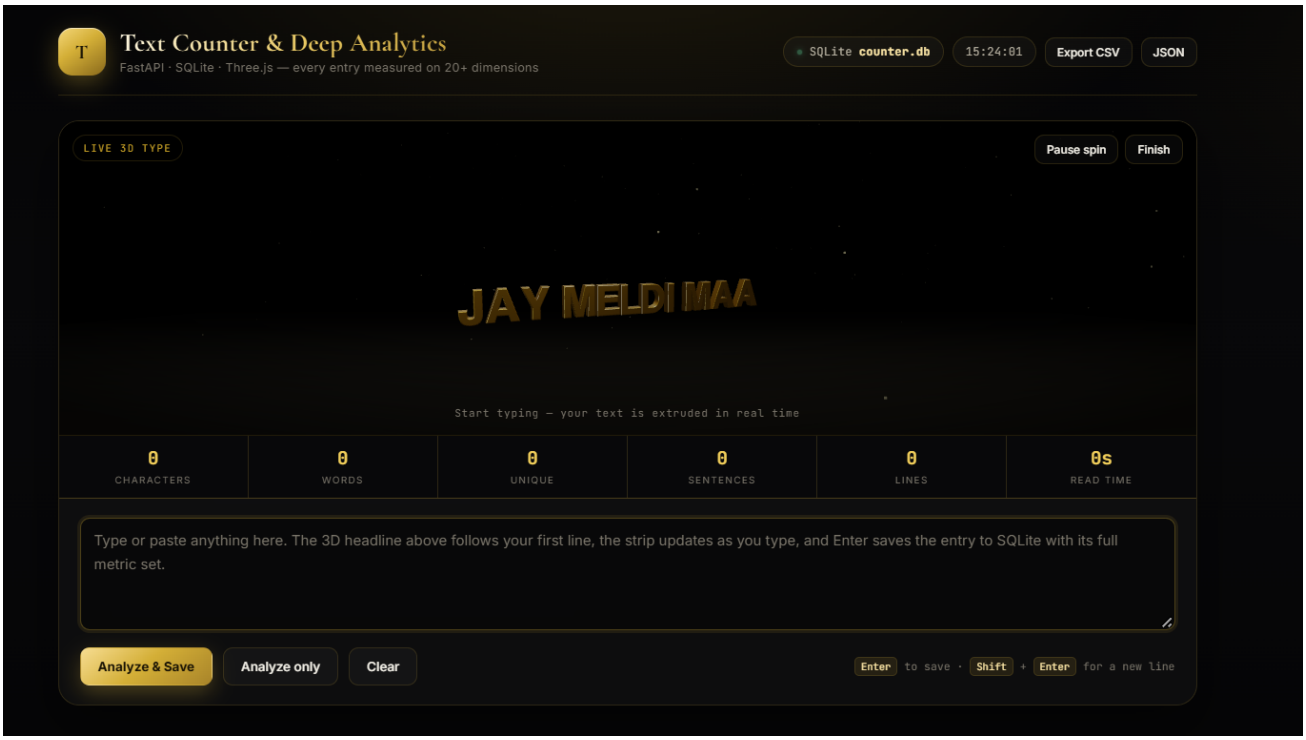
TECHNICAL SPECIFICATION & PROJECT REPORT

3D Text Counter & Deep Text Analytics

A Local-First, High-Performance Text Analytics Dashboard Featuring Real-Time Extruded 3D Typography, 20+ Linguistic Measurements, and Longitudinal Habit Tracking.



Author / Creator: Prince Patel	**Repository:** prinspatel8055/python-text-counter
License: MIT Open Source	**Architecture:** Local-First / Zero Cloud Telemetry
Backend Stack: Python 3.10+ | FastAPI | SQLite	**Frontend Stack:** Three.js WebGL | Vanilla JavaScript
Release: v1.0.0 (Production Stable)	**Target Audience:** Writers, Linguists, Engineers, Researchers

1. Executive Summary & Problem Context

1.1 Executive Summary

3D Text Counter & Deep Text Analytics is a desktop-class, local-first web application engineered to transform standard text counting into a rich analytical and kinetic visual experience. The platform combines **20+ linguistic, compositional, and readability metrics** with real-time, hardware-accelerated **Three.js extruded 3D typography**, while permanently preserving writing history in an embedded, zero-configuration SQLite database.

1.2 The Problem with Traditional Text Counters

Online utility counters typically suffer from four critical deficiencies:

- **Cloud Exposure & Privacy Risks:** Conventional web counters upload user keystrokes and sensitive text to third-party ad networks and proprietary servers.
- **Superficial Metrics:** Standard tools limit analysis to word and character counts, ignoring syllables, lexical diversity, reading speeds, and readability grade levels.
- **Zero Longitudinal Memory:** Session data vanishes upon page refresh. Authors cannot analyze daily productivity trends, active writing hours, or historical vocabulary.
- **Static Interfaces:** Plain, flat interfaces provide no visual engagement during writing.

CORE PRIVACY PRINCIPLE

The Local-First Guarantee: All analytics computations, SQLite persistence, and 3D WebGL rendering execute strictly on 127.0.0.1. No network telemetry, analytics trackers, or external APIs are ever invoked. Your private writing never leaves your computer.

2. System Architecture & Topology

The application is architected according to a strictly decoupled **Three-Tier Modular Architecture**, ensuring zero leakage of business logic across presentation and storage layers.

Tier / Layer	Module File	Technology	Functional Responsibility
Presentation	templates/index.html	Three.js, WebGL, CSS3 Grid	Interactive 3D typography stage, kinetic character animations, responsive charts, live metric updates.
API Controller	app.py	FastAPI, Uvicorn, Pydantic	High-throughput REST API, validation, error handling, auto-browser launch, dynamic port binding, CSV/JSON exports.
Computational Engine	analytics.py	Pure Python (Zero DB deps)	Linguistic tokenization, syllable heuristics, Flesch reading ease score, lexical diversity, stop-word removal, keyword rankings.
Data Persistence	database.py	SQLite3 (WAL Mode)	Relational schema, idempotent migrations, window-based temporal aggregations, full-text date slicing, transaction safety.

3. Deep Text Analytics & Mathematical Models

analytics.py contains a pure functional text-processing engine. It extracts over 20 quantitative linguistic dimensions in sub-millisecond execution times without invoking external cloud APIs.

Category	Metric Name	Algorithmic Definition / Formula
Raw Counts	Characters & Words	Characters with/without whitespace, word count via boundary tokens, sentences split on [.!?]+, lines, paragraphs.
Composition	Character Hierarchy	Classification into letters, numeric digits, whitespace, punctuation, uppercase, lowercase, vowels, consonants, and UTF-8 emojis.
Readability	Flesch Reading Ease	**206.835** – **1.015 × (Total Words / Sentences)** – **84.6 × (Total Syllables / Words)** Clamped to standard bounds [0, 100].
Complexity	Lexical Diversity (TTR)	**(Unique Vocabulary / Total Words) × 100** (Higher % reflects vocabulary breadth).
Acoustic Speed	Reading & Speaking Time	Silent reading benchmarked at **200 WPM**; oral presentation benchmarked at **130 WPM**.
Intelligence	Keyword Extraction	Token frequency analysis stripped of a built-in 130-word English stop-list and filtered for tokens ≥ 3 characters.

Table 3.1: Flesch Reading Ease Complexity Classification

Score Range	Readability Band	Target Audience / Grade Equivalent
90.0 – 100.0	**Very Easy**	5th Grade level (Comics, elementary texts)
80.0 – 89.9	**Easy**	6th Grade level (Conversational prose, simple fiction)
70.0 – 79.9	**Fairly Easy**	7th Grade level (Consumer articles, modern blogs)
60.0 – 69.9	**Standard**	8th – 9th Grade level (Plain English, journalism)
50.0 – 59.9	**Fairly Hard**	10th – 12th Grade level (High school textbooks)
30.0 – 49.9	**Difficult**	College undergraduate papers, technical guides
0.0 – 29.9	**Very Difficult**	Post-graduate treatises, legal briefs, formal specs

3.2 Syllable Heuristic Engine: Syllable calculation operates via tuned vowel clustering: contiguous vowel groups ([aeiouy]+) represent acoustic beats. Words ending in silent -e preceded by a consonant are discounted, words ending in -le are incremented, and every non-empty token is clamped to a minimum syllable count of 1.

4. Real-Time 3D Typography & WebGL Engine

A hallmark innovation of this project is the real-time extruded 3D typography stage, built with **Three.js**. Keystrokes are dynamically translated into true 3D volumetric polygonal geometry rendered through the user's GPU.

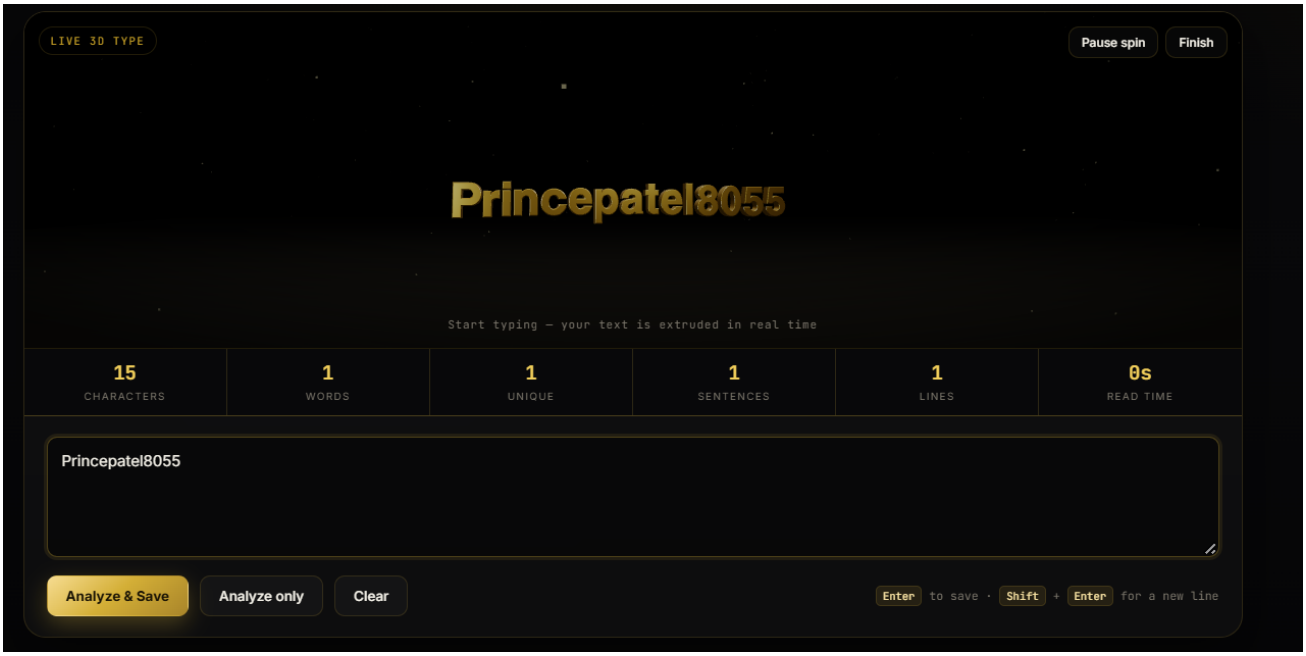


Figure 4.1: Real-time extruded gold 3D geometry dynamically generated as the user inputs characters.

4.1 Key Visual & Kinetic Capabilities:

- **True Extruded TextGeometry:** Glyphs are loaded from JSON typeface definitions and generated as meshes with customizable extrusion depth, beveling, and multi-faceted materials.
- **Incremental Character Diffing:** Instead of tearing down and rebuilding the entire 3D scene per keystroke, the engine diffs the string. New letters drop in with smooth easing, removed letters tumble and fade, and persistent characters continue their natural motion cycle.
- **Physically Based Materials (PBR):** Built using MeshStandardMaterial with tuned roughness and metalness, illuminated by key, rim, and fill light vectors for authentic metallic sheen.
- **Four Metallic Finishes:** Selectable live finishes include **Polished Gold**, **Brushed Brass**, **Champagne Platinum**, and **Antique Bronze**.
- **Fault-Tolerant Fallback:** In environments lacking WebGL support or where font loading is delayed, the stage falls back to extruded polygonal slabs while the rest of the analytics dashboard remains fully functional.

5. Persistence Architecture & Longitudinal Analytics

5.1 Embedded SQLite Schema

Entries and analytics are preserved in an embedded SQLite database (counter.db). SQLite guarantees zero network latency, strict ACID compliance, and zero configuration.

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique sequence identifier.
content	TEXT	NOT NULL	Verbatim raw text string entered by the user.
char_count	INTEGER	NOT NULL	Total character count including whitespace.
word_count	INTEGER	NOT NULL	Total word count generated by the tokenizer.
created_at	TEXT	NOT NULL, ISO-8601	Timestamp recorded in local system time (YYYY-MM-DD HH:MM:SS).

5.2 Temporal Habits & Aggregation Queries

database.py provides optimized queries to evaluate productivity rhythms over time:

- **Day-Wise Writing Velocity:** Tracks entries per day across 7, 30, or 90 days with automatic zero-filling so quiet days appear accurately on charts.
- **24-Hour Circadian Heatmap:** Groups user activity by hour-of-the-day (00:00 to 23:00) to reveal peak creative writing hours.
- **Lifetime Weekday Distribution:** Aggregates activity across Monday through Sunday.

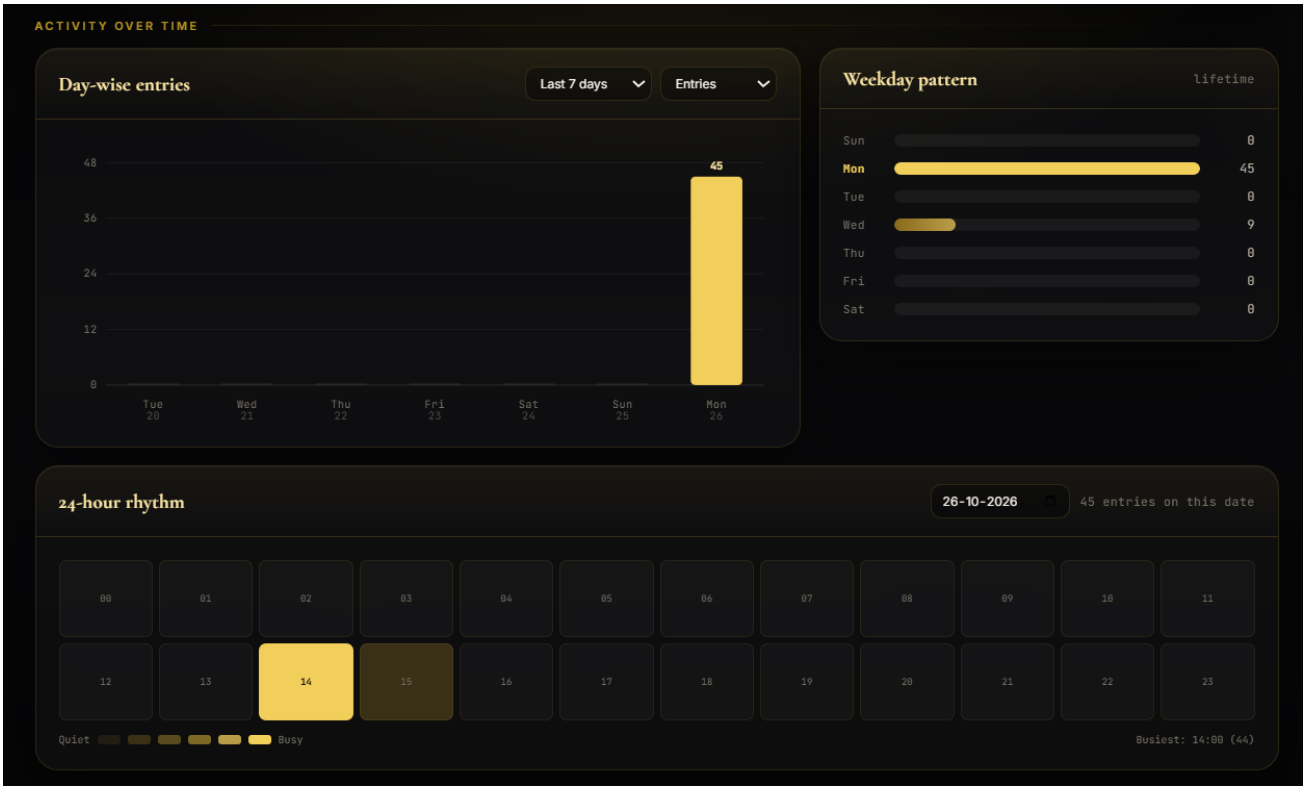


Figure 5.1: Longitudinal writing habit charts — Day-wise velocity, Weekday distribution, and 24-Hour circadian rhythm.

6. RESTful API Specification

The backend exposes a clean, self-documenting REST API developed with **FastAPI**. Interactive Swagger documentation is automatically served at /docs.

HTTP Method	Endpoint Path	Parameters / Body	Response Description
GET	/	None	Serves the single-page application dashboard.
GET	/api/health	None	Returns system health, app version, and SQLite connectivity status.
POST	/api/analyze	{"text": str}	Calculates full 20+ linguistic metrics without persisting the entry.
POST	/api/texts	{"text": str}	Calculates metrics, saves to SQLite, and returns stored record with ID.
GET	/api/texts	search, limit, sort, date	Filtered, paginated list of saved text entries.
GET	/api/texts/{id}	Path: id (int)	Retrieves specific entry with recomputed metrics and keywords.
DELETE	/api/texts/{id}	Path: id (int)	Permanently deletes an individual entry by ID.
DELETE	/api/texts	Query: confirm=true	Wipes all stored history after explicit affirmative confirmation.
GET	/api/stats/overview	Query: days=7|30|90	Consolidated payload containing KPIs, daywise, weekday, and keywords.
GET	/api/export	Query: format=csv|json	Streams full historical dataset as a downloadable CSV or JSON attachment.

Sample JSON Request & Response:

```
POST /api/analyze HTTP/1.1 Content-Type: application/json {"text": "The quick brown fox jumps over the lazy dog."} HTTP/1.1 200 OK { "counts": {"chars": 43, "chars_no_spaces": 35, "words": 9, "unique_words": 8, "sentences": 1, "syllables": 11}, "readability": {"flesch_reading_ease": 83.3, "band": "Easy"}, "timing": {"reading_time_sec": 2.7, "speaking_time_sec": 4.1}, "keywords": [{"word": "quick", "count": 1}, {"word": "brown", "count": 1}, {"word": "jumps", "count": 1}] }
```

7. Interface Gallery: Metrics & Analytics

The user interface is designed with a dark luxury aesthetic, high information density, and instant responsiveness to keystrokes.

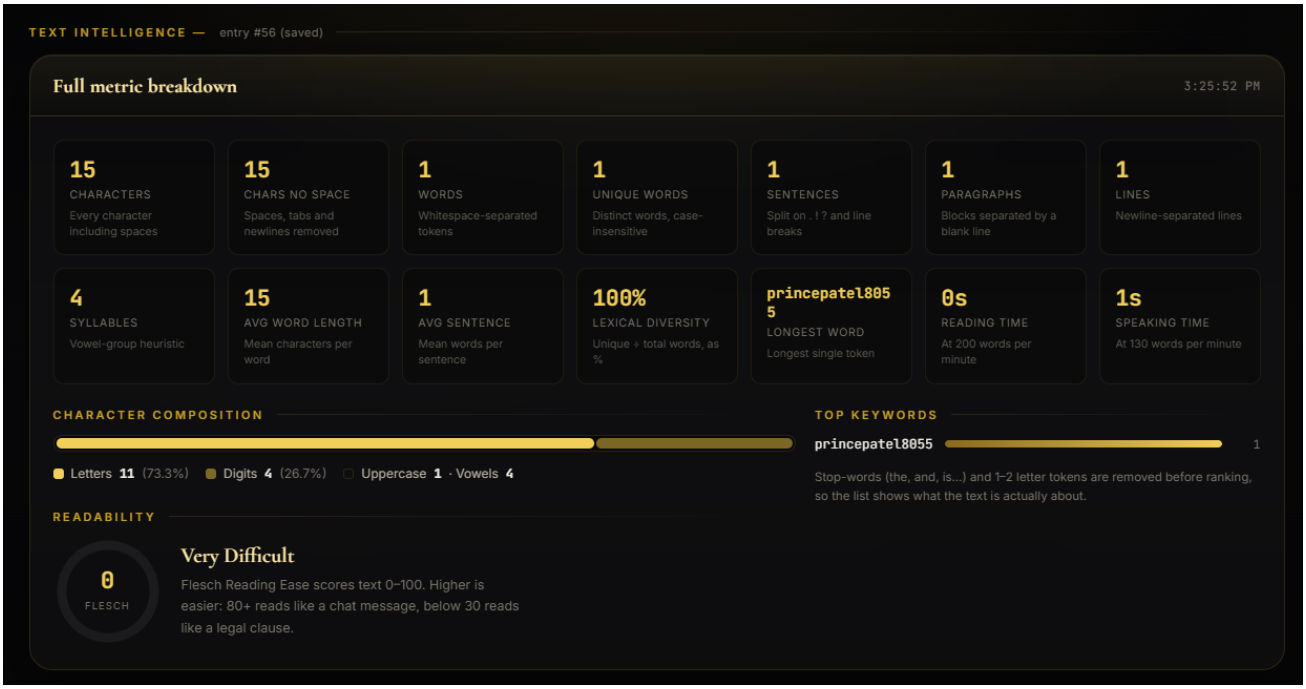


Figure 7.1: Comprehensive Text Intelligence Panel displaying counts, composition, reading times, and Flesch readability.

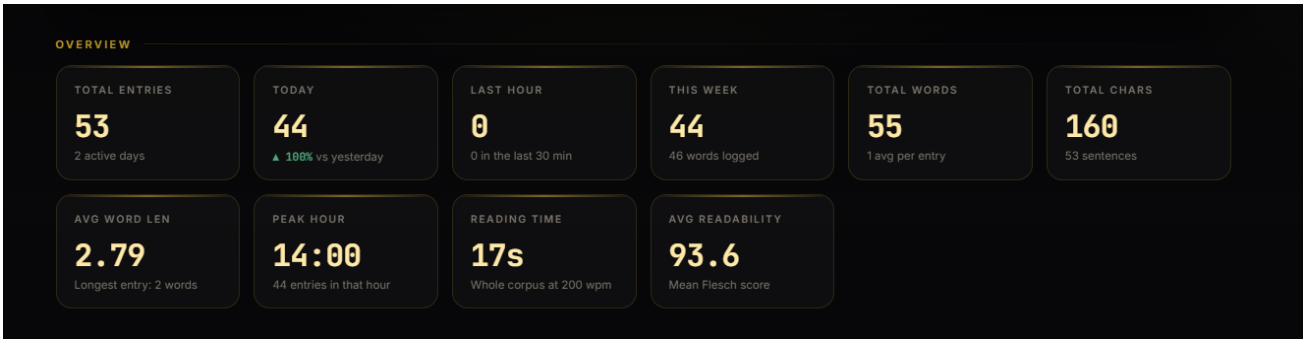


Figure 7.2: Overview KPI cards showing total saved entries, cumulative word count, peak writing hours, and average readability.

8. Data Management & Installation Guide

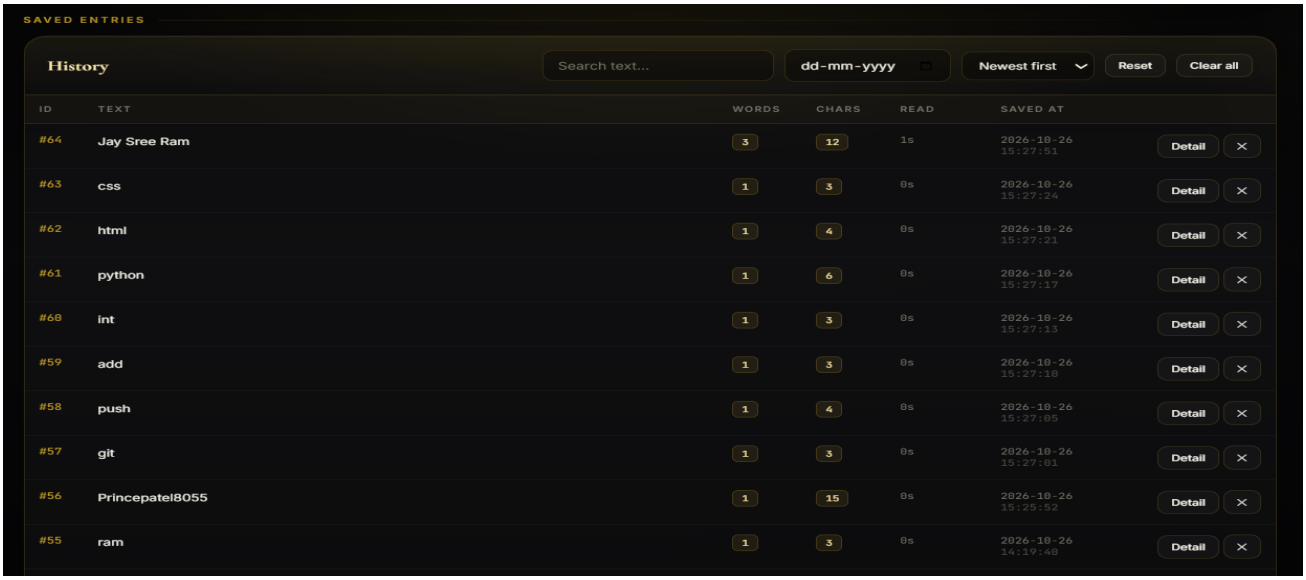


Figure 8.1: Searchable history data table with multi-criteria sorting, date filters, deletion safeguards, and CSV/JSON export.

8.1 One-Click Setup & Platform Execution:

Pre-configured executable launch scripts handle virtual environment detection, dependency installation, dynamic port allocation, and automated browser opening:

Platform	Launch Command	Internal Script Sequence
Windows	Double-click run.bat	Detects python, activates/creates .venv, runs pip install -r requirements.txt, and launches python app.py.
macOS / Linux	./run.sh	Grants execution permissions (chmod +x), configures virtualenv, and boots FastAPI.
Manual CLI	python app.py	Standard Python launch. Scans for an open port starting at 8000 and triggers the default browser.

8.2 Keyboard Ergonomics: Enter saves and analyzes the entry; Shift + Enter inserts a newline.

9. Roadmap, Community & Star Support

9.1 Engineering Roadmap:

- **v1.1:** Integration of language-detection models with multilingual stop-words (Spanish, German, Hindi, Gujarati).
- **v1.2:** Native Dark/Light theme switching and customizable 3D stage lighting controls.
- **v1.3:** Automated unit test suite with 100% branch coverage across analytics.py.
- **v2.0:** Optional multi-user authentication module for shared classroom and enterprise team deployments.

SUPPORT & GITHUB STARS

SUPPORT THIS OPEN SOURCE REPOSITORY ON GITHUB:
If you find this project valuable for your writing workflow, research, or development learning, please consider starring the repository on GitHub! Stars help the open-source community discover high-quality, local-first tools.

GitHub Repository: https://github.com/prinspatel8055/python-text-counter
Issue Tracker: https://github.com/prinspatel8055/python-text-counter/issues

Project Creator & Maintainer:
Prince Patel (@prinspatel8055)
Creator of 3D Text Counter & Deep Text Analytics
Engineering elegant local-first software with Python, FastAPI, SQLite, and Three.js.

Document Information: *Generated automatically from the project codebase. Copyright © 2026 Prince Patel. Distributed under the terms of the MIT License.*